

P3159R0

C++ Range Adaptors and Parallel Algorithms

Published Proposal, 2024-03-17

Author:

[Bryce Adelstein Lelbach \(he/him/his\)](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Optimizations

2.1 Non-Trivial Removal

2.2 Non-Trivial Grouping

2.3 Trivial Removal and Grouping

3 Cheatsheet

§ 1. Introduction

Range factories and adaptors are a powerful compositional tool that allow users to elegantly express non-modifying transformation, filtering, grouping, and reshaping of a range without materializing the changes in global memory. Adaptors and factories produce range views, which are non-owning and lazily-evaluated ranges. Due to their lazy nature, parallelizing views presents challenges. Views apply their transformative logic one element at a time on-demand. But, for parallelization, we need to extract that transformative logic and apply it in bulk across the entire underlying range.

To implement parallel algorithms that consume ranges, we must build a library-only optimizing kernel builder that recursively decomposes arbitrary ranges (and iterators to them) into the pipeline of adaptors and factories that constructed them. This kernel builder may then substitute each adaptor and factory for a parallel-friendly alternative and/or insert in-place preprocessing passes as needed.

```
auto optimize_range(range auto&& rng) {
    if constexpr(has_base(rng)) {
        // Dispatch to logic that optimizes this adaptor.
        return optimize_range(rng.base());
    } else {
        return rng;
    }
}
```

If we restrict ourselves to the closed set of adaptors and factories in the C++ standard library, then for any given range:

- We can determine if it came from an adaptor or factory.
- We can determine what adaptor or factory it came from.
- We can get the base ranges (if any) from it.
- We can get any of the user-provided parameters to the adaptor or factory that we may need.

§ 2. Optimizations

§ 2.1. Non-Trivial Removal

Certain range adaptors remove elements in a way that cannot be trivially computed a priori (ex: `filter`, `take_while`).

Parallel algorithm implementations may distribute the input among threads a priori, creating N execution agents, each of which will process M initial elements. Since we cannot compute which elements will be removed during this distribution, each execution agent will need to start with M initial elements and later discard any of those elements that meet the removal criterion.

When consuming these removing adaptors in parallel, they need to be substituted into range adaptors that instead replace those elements with tombstones. A tombstone is an element of a range that represents non-existent data that needs to be removed from the range. A tombstone is represented with an optional-like object.

The tombstones must be removed when the range is consumed by a parallel algorithm. The simplest approach is to insert a `copy_if` preprocessing pass. This must be done in-place to avoid materializing the adapted range in memory. Such a pass is known as stream compaction

```
for_each(rng | filter(f), g);

void kernel(range auto&& rng0) {
    rng1 = compact(rng0 | filter_tombstone(f));
    for_each_collective(rng1);
}
```

Stream compaction is often implemented with a scan.

```
auto copy_if(range auto&& in, output_iterator auto out, auto pred) {
    vector<uint8_t> flags(size(in));

    transform(par, in, begin(flags), pred);

    vector<size_t> indices(size(in));

    exclusive_scan(par, flags, begin(indices), 0);

    for_each(par, zip(in, flags, indices),
        apply([&in] (auto e, auto flag, auto index) {
            if (flag) out[index] = e;
        }));

    return subrange(begin(out), next(out, indices.back()));
}
```

An alternative approach is to defer removal of the tombstones, instead wrapping all subsequent adaptors, user-provided operations, and the parallel algorithm implementation to ignore the tombstones. In some cases, it will never be necessary to insert a stream compaction pass, because everything can simply be wrapped:

```
for_each(rng | filter(f), g);

for_each(rng | transform([f] (auto x) {
    if (!f(x)) return tombstone(x); else return nullopt;
}),
    [g] (auto x) { if (x) g(*x); });

reduce(rng | filter(f), g);

reduce(rng | transform([f] (auto x) {
    if (!f(x)) return tombstone(x); else return nullopt;
}),
    [g] (auto l, auto r) {
        if (l && r) return g(l, r);
    });
```

```

    else if (l) return l;
    else if (r) return r;
    else      return {};
});

```

However, not all adaptors and parallel algorithms can be wrapped to ignore tombstones. For example, some adaptors like `adjacent` and `enumerate` are position-aware, meaning they either access neighboring elements or care about the position of the element in the range. If tombstones are present, they will throw off the position logic. `adjacent` needs to give you adjacent non-tombstone elements and `enumerate` needs to not count the tombstones, which would require the sort of lazy filtering logic that we seek to avoid.

To defer removal of tombstones, the optimizing kernel builder must keep track of whether the reconstructed range is tombstoned as it recurses through it. If it encounters an operation that requires tombstoning, it must mark the reconstructed range as tombstoned. If it encounters an operation that cannot be wrapped to ignore tombstones, then it must insert a stream compaction pass and mark the reconstructed range as untombstoned. Maintaining this state adds complexity. A pipeline may enter and exit the tombstone state multiple times.

```

sort(rng | filter(f) | transform(g) | adjacent<2> | filter(h) | transform(i));
// ~~~~~ XXXXXXXXXXXX ~~~~~
//      Tombstoned      Untombstoned      Tombstoned

void kernel(range auto&& rng0) {
    rng1 = compact(rng0 | filter_tombstone(f) | transform_tombstone(g));
    rng2 = compact(rng1 | adjacent<2> | filter_tombstone(h) | transform_tombstone(i)
    sort_collective(rng2);
}

```

Deferred removal of tombstones offers potential performance benefits by avoiding unnecessary or redundant stream compaction passes, at the cost of additional complexity. For the sake of simplicity, we propose to not implement deferred removal at this time, and instead immediately inserting the stream compaction preprocessing pass when encountering range adaptors that perform non-trivial removal.

§ 2.2. Non-Trivial Grouping

Certain range adaptors group or combine elements in a way that cannot be trivially computed a priori (ex: `chunk_by`, `split`). These adaptors reduce the size of the sequence. When considering how to parallelize them, we should think of them as both transforming and removing elements. A parallel implementation of these adaptors must do two things:

- Compute the groupings, which requires some form of summation.
- Discard whichever of the M initial elements per execution agent do not represent one of the groupings.

The computation of the groupings can be implemented by inserting an in-place scan preprocessing pass.

```
for_each(rng | chunk_by(f) | adjacent<2>, g);

void kernel(range auto&& rng0) {
    rng1 = grouping(rng0);
    for_each_collective(rng1 | adjacent<2>, g);
}
```

As discussed in the Non-Trivial Removal section, the removal of elements can be done by inserting an in-place `copy_if` preprocessing pass, and `copy_if` can be implemented with a scan. These two operations can be combined into a single scan-based preprocessing pass that both computes the groupings and removes all but one element per grouping.

```
auto chunk_by(range auto&& in, auto out, auto pred) {
    vector<uint8_t> flags(size(in));

    transform(in | adjacent<2>, begin(flags),
        apply([&] (auto l, auto r) { return pred(l, r); }));

    struct interval {
        bool flag;
        size_t index;
        size_t start;
        size_t end;
    };

    vector<interval> intervals(size(in));

    exclusive_scan(par,
        flags | transform([] (auto b) { return interval{b, 0, 0, 1}; }),
        begin(intervals),
        interval{true, 0, 0, 1},
        [] (auto l, auto r) {
            return interval{r.flag,
                r.flag ? l.index + r.index : l.index + r.index + 1,
                r.flag ? l.start + r.start : l.end,
                l.end + r.end};
        });

    for_each(par, zip(flags, intervals),
        apply([&] (auto flag, auto i) {
            if (!flag)
                out[i.index] = subrange(next(begin(in), i.start),
                    next(begin(in), i.end));
        }));
}
```

```

    return subrange(out, next(out, intervals.back().index));
}

```

§ 2.3. Trivial Removal and Grouping

Certain range adaptors remove elements in a way that can be trivially computed a priori (ex: `drop`, `stride`) or group or combine elements in a way that can be trivially computed a priori (ex: `chunk`, `adjacent`). Thus, when consuming these adaptors in parallel, we can account for the removal or grouping while doing work distribution.

```

for_each(rng | drop(X), f);

auto start = begin(rng) + X;
auto end = end(rng);
for_each(start, end, f);

```

However, if these range adaptors adapt a range that contained a non-trivial removal or grouping adaptor, then they become non-trivial as well, and need to be handled with an in-place stream compaction pass.

There's two approaches to handling trivial removals and groupings: 0) Always insert an in-place scan pass and don't handle them during work distribution. This has the advantage of simplicity. 1) Only insert in-place scan pass if there is a prior non-trivial removal or grouping, and otherwise handle it during work distribution. This requires maintaining some state when reconstructing the range, but is more efficient.

We propose to do (1).

§ 3. Cheatsheet

Range Adaptor or Factory	Iterator Category	Internal Iteration?	Output Size Unknown?	Position Aware?	Reshapes?	Optimization
<code>ref_view</code>	Contig					
<code>owning_view</code>	Contig					
<code>all</code>	Contig					
<code>common</code>	Contig					
<code>as_const</code> <code>as_rvalue</code>	Contig					
<code>iota</code>	RA					
<code>transform</code>	RA					
<code>elements</code>	RA					

keys values	RA					
enumerate	RA			Yes		
reverse	RA					
zip zip_transform	RA			Yes		
cartesian_product	RA			Yes		
filter	Forward	Yes	Yes			Non-Trivial Removal
take	Contig			Yes		Trivial Removal
take_while	Contig	Yes	Yes	Yes		Non-Trivial Removal
drop	Contig			Yes		Trivial Removal
drop_while	Contig	Yes	Yes	Yes		Non-Trivial Removal
join join_with	BiDi	Yes			Yes	?
split lazy_split	Forward	Yes	Yes	Yes	Yes	Non-Trivial Grouping
chunk	Forward	Yes		Yes	Yes	Trivial Grouping
chunk_by	Forward	Yes	Yes	Yes	Yes	Non-Trivial Grouping
adjacent adjacent_transform	RA	Yes		Yes	Yes	Trivial Grouping
slide	RA	Yes		Yes	Yes	Trivial Grouping
stride	RA	Yes		Yes		Trivial Removal